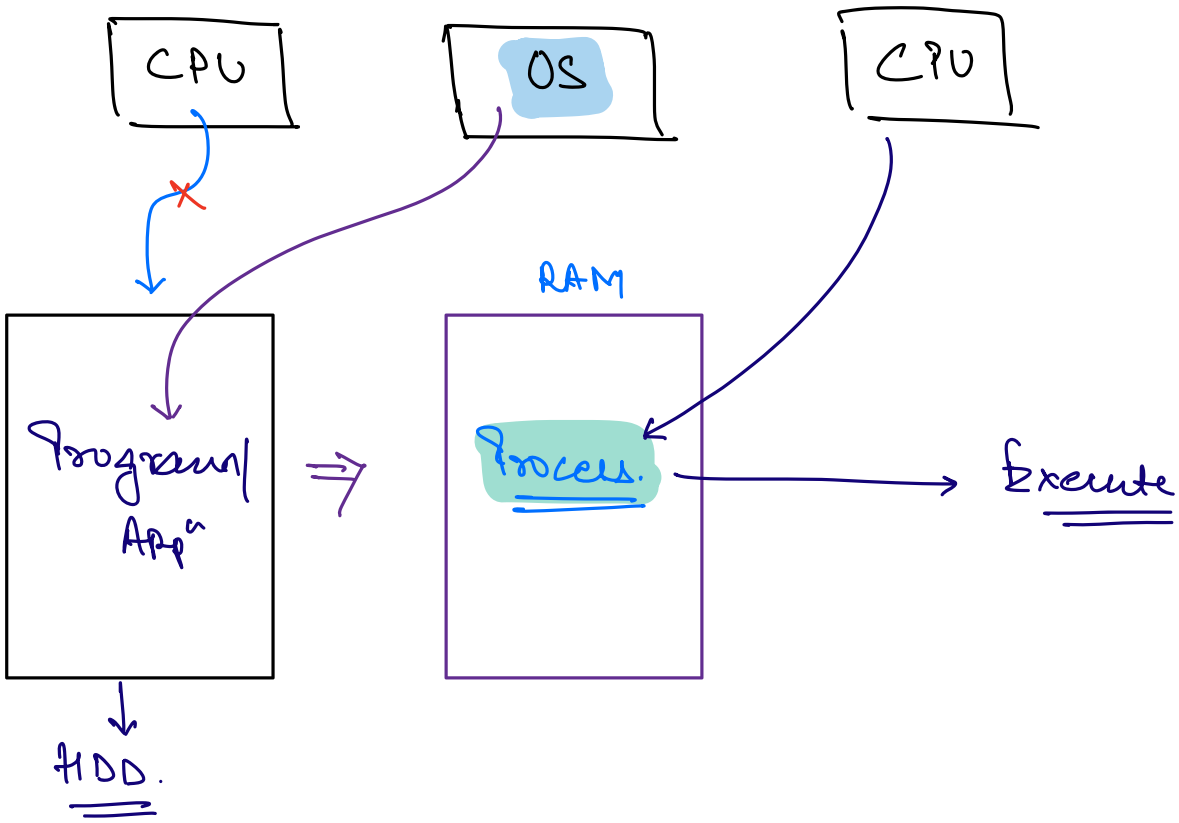
Agenda.

- → Process
- → Threads
- → Single Core (vs) Multi Core
- → Concurrency (vs) Parallelism
- → Coding.

Process.



Can CPU talk to the HDD ?

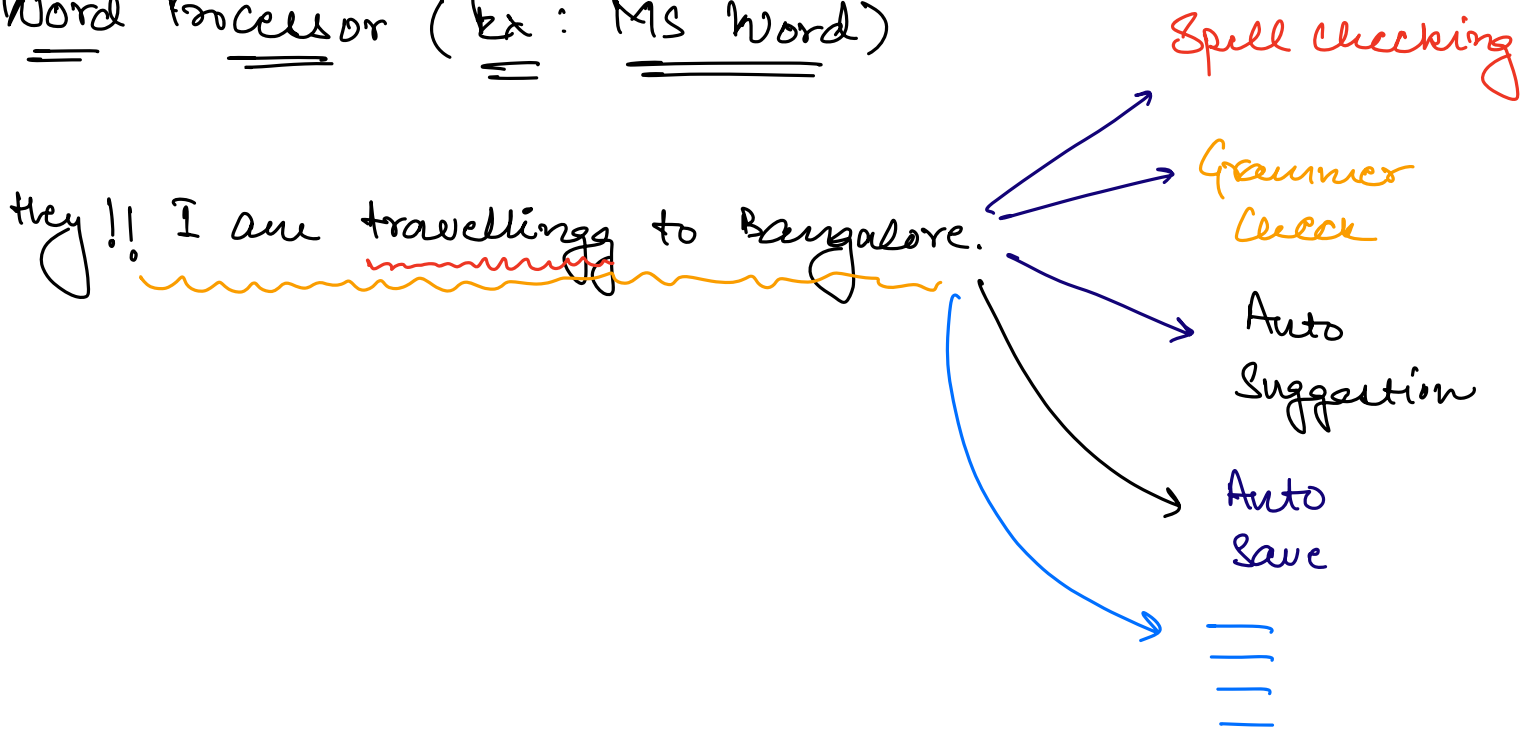
⇒ No, because of huge speed difference b/w CPU & HDD.

Very fast ← CPU & HDD. → Very slow

⇒ OS picks up the program from HDD and put it into the RAM & it becomes process. CPU picks up the picks the process from RAM & execute it.

⇒ Process : Program in Execution.

Word Processor (Ex : MS Word)



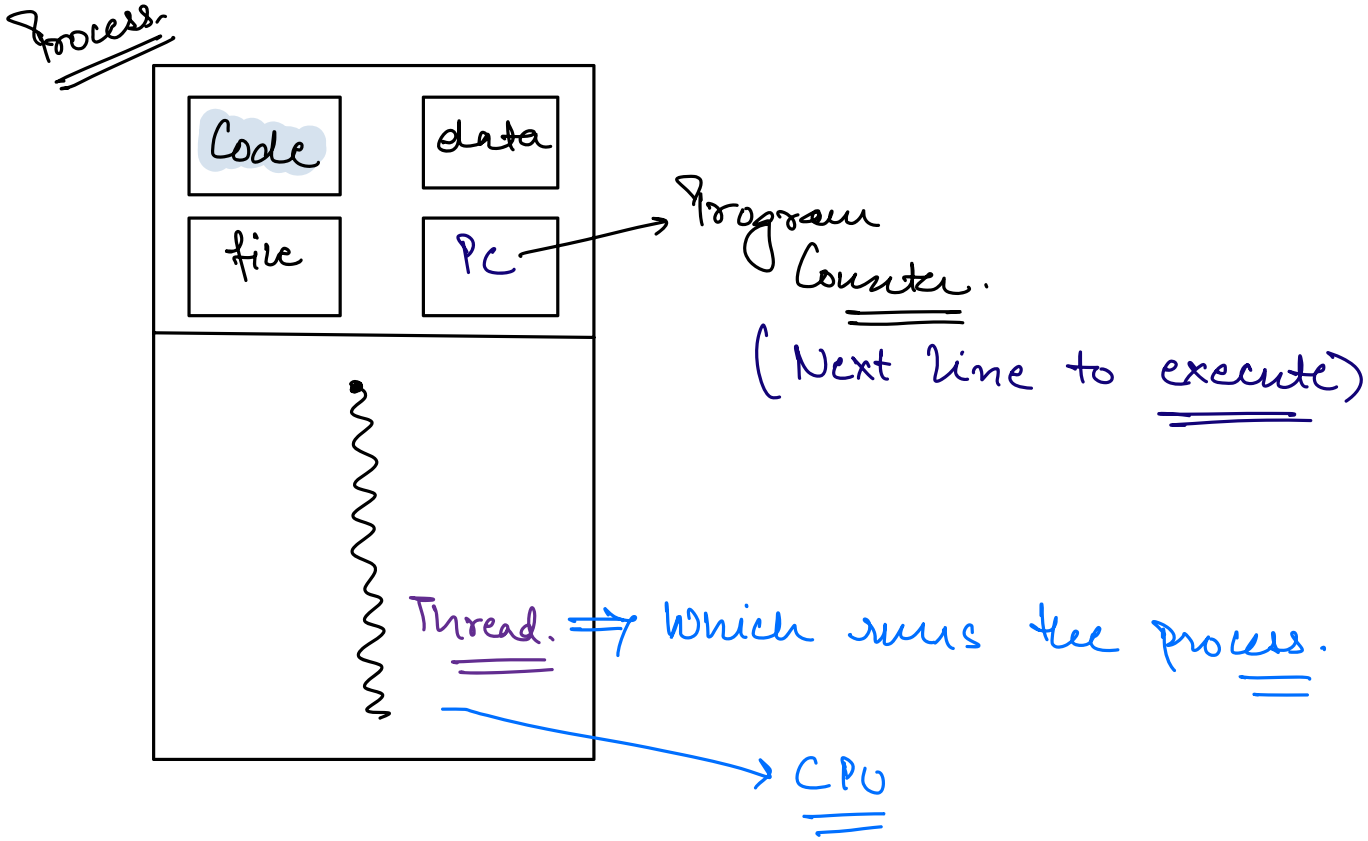
Thread

- Unit of CPU execution.
- CPU executes the Thread.
- Whenever anything is running on our computer, there is a CPU running a thread.

Core

```
main() {
    cout("Hello World");
}
```

⇒ Default thread gets created.
(main thread).

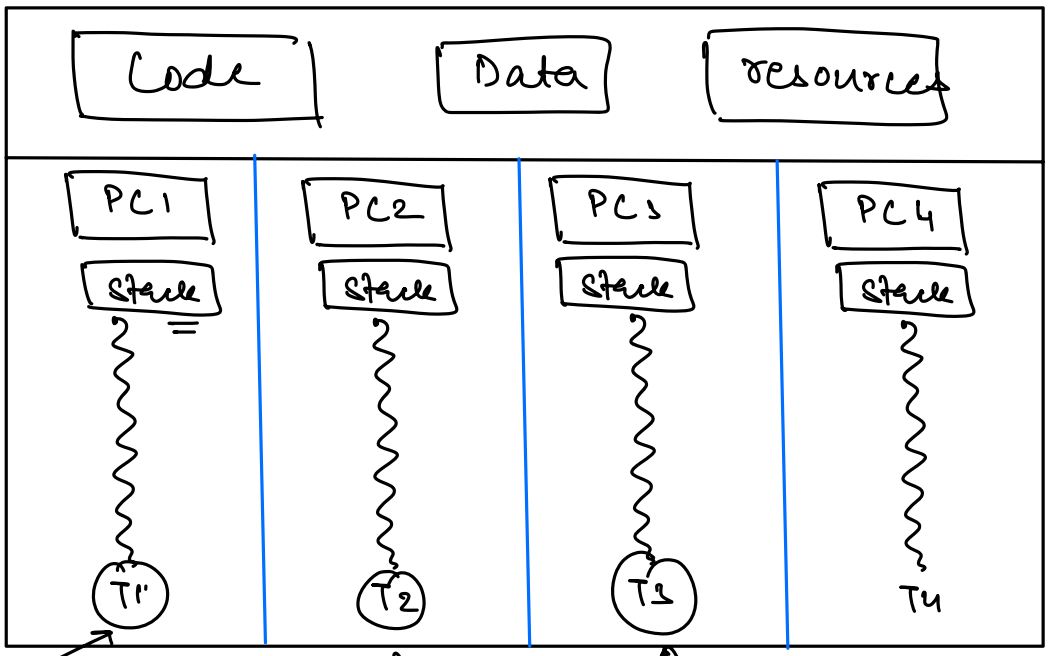


⇒ To get executed, every app^n should have atleast one thread.

Process

PCB

Process Control Block



Auto Suggestion

Auto Saving

Spell Check

- - -

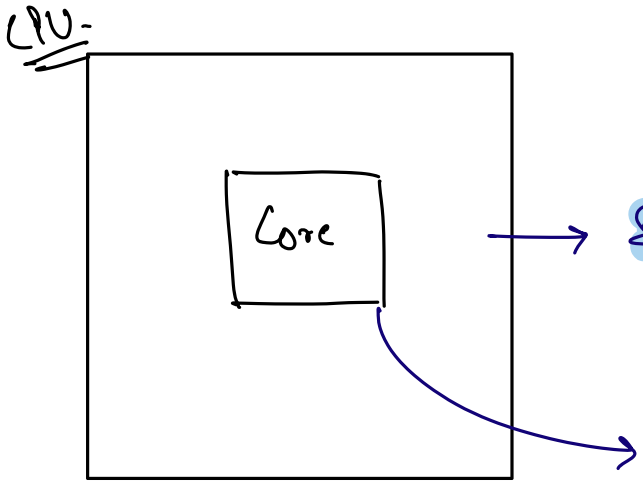
A process can have any no of threads.

Process takes more memory than a thread.

Creating a process takes more time & space.

Single Core vs Multi Core CPU

Core → Execution Unit of CPU.



→ Single Core

A thread gets assigned to the core, which actually does the work.

Speed of the core ~ 1GHz

1×10^9 operations / sec.

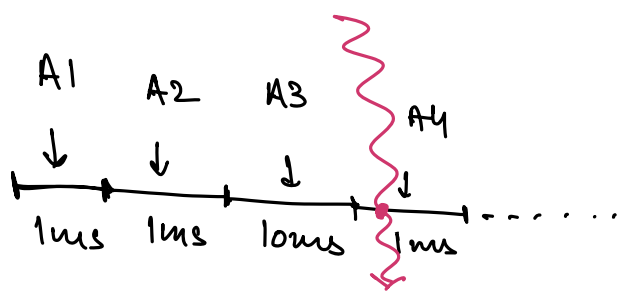
Single Core

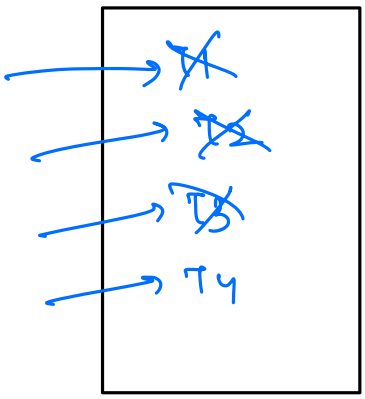
10^9 operations ⇒ 1sec

10^6 operations ⇒ $\frac{1}{10^9} \times 10^6$

10^6 → 1M

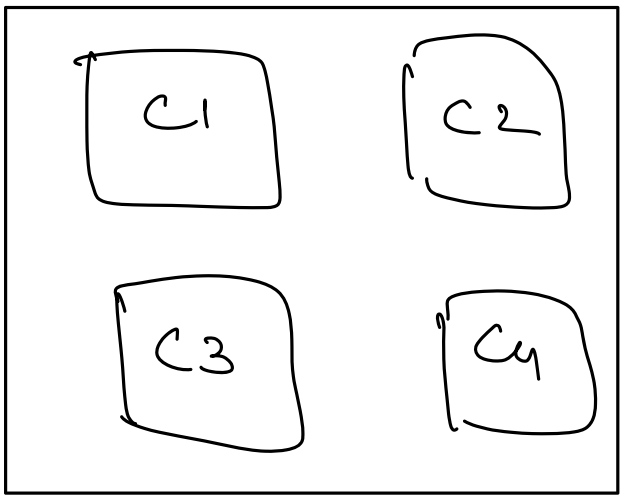
$= \frac{1}{1000}$ sec.

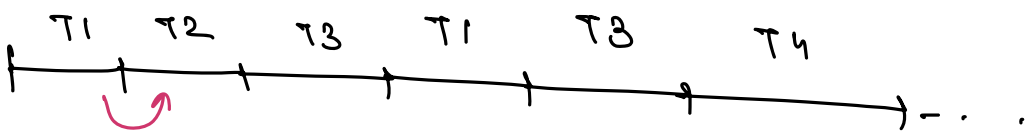




Single Core

Quad Core





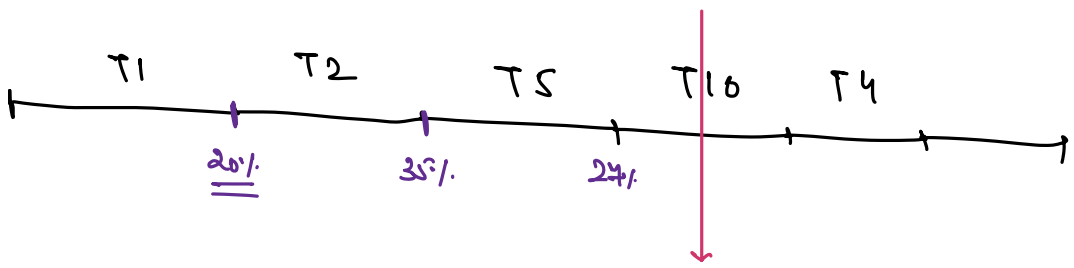
Context Switching

Concurrency vs Parallelism.

↓

When a system can have multiple threads in different stages of execution at the same time but NOT necessarily making the progress in parallel.

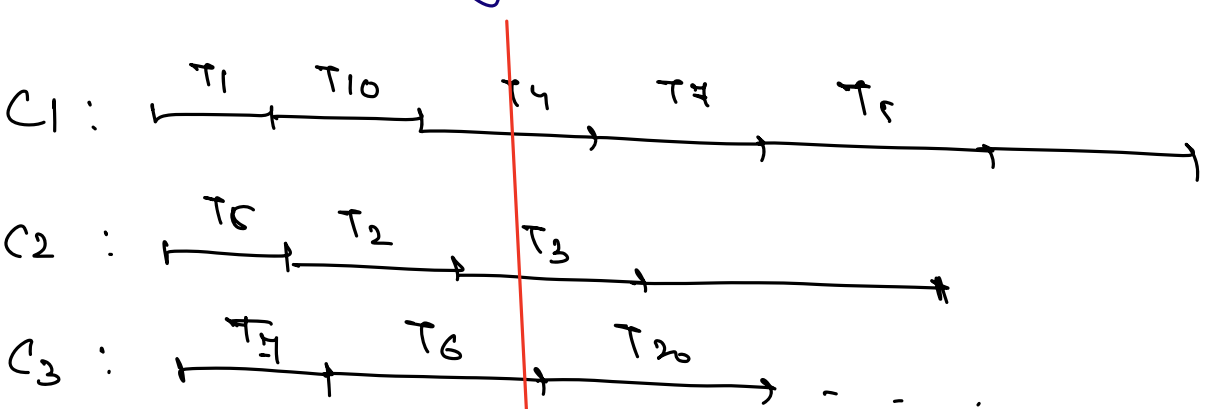
Single Core : C1



T1 : 20%.
T2 : 35%.
T5 : 27%.
T10 : 8%.

Parallelism.

↳ Concurrency + Progress at the same time.



↳ Multiple Cores.

Print Hello World from a separate thread.

I) Create a task (class) that we want to execute in a separate thread.

```
Class HelloWorldPrinter {

}
```

II) Make the task class, implement the Runnable interface.

```
Class HelloWorldPrinter implements Runnable {

}
```

III) Implement run() method.
↳ Write the code that we want to execute in a separate thread.

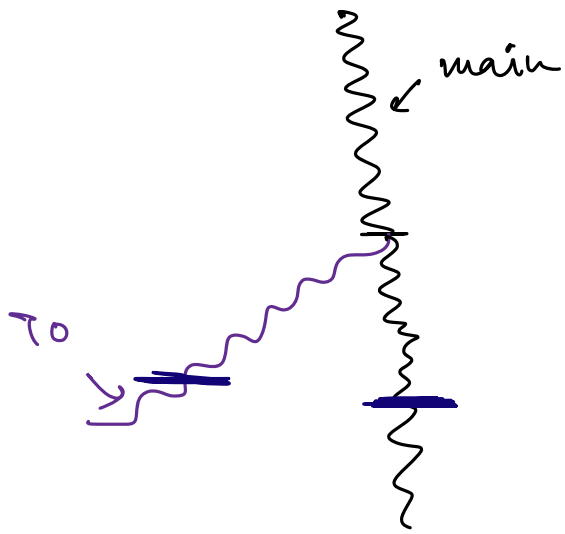
```
Class HelloWorldPrinter implements Runnable {

    Void run() {
        Sout ("Hello World");
    }

}
```

IV) Create the thread and assign the task to thread.

```
HelloWorldPrinter task = new HelloWorldPrinter();
Thread t = new Thread(task)
```

V) Start the thread.

```
t.start()
```



Q. Print no's from 1 to 100, each from a separate thread.

HW.